

Les fixtures : la vraie stratégie

Extrait du livre *Maîtriser JavaScript, TypeScript et Playwright* — Partie 13, Architecture d'un framework Playwright. Daura Rady — version de travail, août 2026.

La plupart des frameworks Playwright que j'ai repris en mission avaient des fixtures. Presque aucun ne s'en servait correctement. On y trouvait un `test.extend` qui instanciat trois Page Objects, et tout le reste du setup empilé dans des hooks.

Ce chapitre explique pourquoi ce découpage coûte cher, et ce qu'une fixture bien conçue permet réellement.

1. Ce qu'est vraiment une fixture

Les fixtures Playwright sont un système d'**injection de dépendances**. Si vous venez de Spring ou d'Angular, c'est le même principe : le composant déclare ce dont il a besoin, le framework le résout, l'instancie, l'injecte et le nettoie.

Le test ne crée plus ses dépendances. Il les reçoit.

```
test('should update profile', async ({ page, testUser }) => {  
  // page et testUser existent déjà, créés pour ce test uniquement  
});
```

La conséquence est structurelle : vos tests sont isolés par construction, pas par discipline. Il n'y a plus d'état partagé qu'on aurait oublié de réinitialiser, parce qu'il n'y a plus d'état partagé du tout.

2. Le problème que ça résout

Voici le pattern qu'on trouve dans neuf projets sur dix. Il fonctionne, et c'est bien ça le problème : il fonctionne jusqu'au moment où il ne fonctionne plus, et là personne ne comprend pourquoi.

```
let userId: number;

test.beforeEach(async ({ request }) => {
  const res = await request.post('/api/users', { data: { email: 'test@test.local' } });
  userId = (await res.json()).id;
});

test.afterEach(async ({ request }) => {
  await request.delete(`/api/users/${userId}`);
});
```

Quatre défauts, par ordre de gravité croissante.

La variable partagée. `userId` vit au niveau du fichier. Tant que les tests s'exécutent séquentiellement dans leur fichier et que cette variable n'est pas réutilisée de manière concurrente, le problème peut rester invisible — un worker exécute plusieurs tests l'un après l'autre, ce n'est pas un worker par test. Mais cette donnée vit hors du cycle de vie du test, ce qui rend le comportement fragile dès que l'organisation de la suite évolue : un `test.describe.serial` ajouté, l'ordre d'exécution changé, une assertion déplacée, et la variable devient une bombe à retardement.

Le setup s'exécute toujours. Sur un fichier de cinquante tests dont dix ont besoin d'un utilisateur, le `beforeEach` en crée cinquante. Quarante appels API inutiles par run, multipliés par le nombre de fichiers.

Setup et teardown sont à deux endroits. Pour comprendre le cycle de vie d'une donnée il faut lire deux blocs séparés par cinq cents lignes de tests. Six mois plus tard, quelqu'un supprime le `afterEach` en pensant qu'il ne sert à rien.

Rien n'est typé. `userId` est un `number` parce qu'on l'a déclaré ainsi, pas parce que l'API le garantit.

3. Le mécanisme de `use()`

Il n'y a qu'une seule chose à comprendre pour les fixtures, et c'est celle-là.

```
testUser: async ({ userService }, use) => {
  const user = await userService.create(); // AVANT le test
  await use(user);                        // le test s'exécute ICI
  await userService.delete(user.id);      // APRÈS le test
},
```

`use()` est le moment où vous prêtez l'objet au test. Tout ce qui précède est la préparation, tout ce qui suit est le rangement. Le test entier s'exécute pendant cet `await`.

Une fixture reçoit trois arguments :

Argument	Rôle
<code>{ page, userService }</code>	Les dépendances, c'est-à-dire d'autres fixtures
<code>use</code>	La fonction qui passe la valeur au test
<code>testInfo</code> (optionnel)	Les métadonnées du test en cours

Les dépendances sont elles-mêmes des fixtures. C'est ce qui rend le système composable : vous empilez des briques, Playwright résout l'arbre.

💡 Si tu débutes avec les fixtures, retiens seulement ceci :

- une fixture prépare une dépendance,
- `use()` la donne au test,
- le code après `use()` nettoie,
- elle n'est créée que lorsqu'on la demande,
- garde tes données métier en scope test.

Tout ce qui suit — scopes worker, fixtures auto, options paramétrables, ordre d'exécution précis, `mergeTests` ... — est le niveau avancé. Reviens-y une fois ce socle acquis.

La conséquence directe : le lazy loading

Une fixture n'est instanciée que si un test la demande. C'est la vraie différence avec un hook, et elle est mesurable. Sur l'exemple précédent, on passe de cinquante créations d'utilisateur à dix. Sur une suite de régression complète, c'est plusieurs minutes de CI et autant de charge en moins sur l'environnement de test.

Règles de nommage

Un nom de fixture commence par une lettre ou un underscore, et ne contient que des lettres, des chiffres et des underscores.

- ✗ `2ndPage`, `my-page`, `page.admin`
- ✓ `secondPage`, `myPage`, `pageAdmin`, `_helperPage`

4. Le teardown est garanti, votre setup ne l'est pas

Il faut lever tout de suite un contresens que je vois régulièrement, y compris dans des articles bien notés : **non, un test qui échoue ne fait pas échouer `use()`**, et le code de teardown n'a pas besoin d'être protégé pour s'exécuter.

La documentation est explicite : chaque fixture a une phase de setup avant l'appel à `use()` et une phase de teardown après, et le teardown s'exécute dès que la fixture n'est plus utilisée. Le code du runner va plus loin : le teardown est garanti même si le test échoue ou dépasse son timeout, tant qu'il reste du temps dans le budget de teardown du worker.

```
testUser: async ({ userService }, use) => {
  const user = await userService.create();
  await use(user);
  await userService.delete(user.id); // s'exécute même si le test échoue
},
```

Ce code est correct. C'est même la forme que donne la documentation. Si vous ne retenez qu'une chose du mécanisme des fixtures, retenez celle-là : **la garantie de nettoyage est une propriété du runner, pas une propriété de votre écriture.**

Nuance à garder en tête : ce n'est pas une garantie transactionnelle absolue. Le teardown de fixture est un cycle de vie géré par Playwright tant que le processus worker peut encore exécuter du code — pas une promesse que le nettoyage survit à n'importe quelle circonstance. Les trois cas ci-dessous en sont les limites concrètes.

Les trois cas où la garantie ne joue pas

Le worker meurt. Crash du processus, mémoire épuisée, job CI annulé. Aucun code JavaScript ne s'exécute après. C'est irréductible côté framework, et ça se traite ailleurs : une routine de purge périodique sur l'environnement de recette, ou un préfixe de nommage qui permet de nettoyer en masse.

Le budget de teardown est épuisé. Le temps de setup et de teardown d'une fixture compte dans le timeout du test. Une fixture lente peut donc faire expirer le test, puis se voir couper pendant son propre nettoyage. Voir la section 9.

La fixture échoue avant d'atteindre `use()`. C'est le cas le plus fréquent et le seul que vous pouvez traiter dans votre code. Une fixture qui n'a jamais été résolue n'a pas de teardown à exécuter.

Le vrai rôle de `try / finally`

Considérez une fixture qui construit une donnée en plusieurs étapes, ce qui est la norme dès qu'on dépasse le CRUD :

```
const user = await userService.create({ email });
await userService.validateKyc(user.id); // peut échouer
await walletService.deposit(user.id, 10_000); // peut échouer
await use(user);
await userService.delete(user.id);
```

Si `validateKyc` lève une exception, l'exécution n'atteint jamais `use()`. Playwright marque la fixture en échec et saute le test, mais l'utilisateur créé à la première ligne existe toujours en base. La garantie du runner ne s'applique pas, parce qu'il n'y a rien à démonter d'une fixture qui n'a jamais été livrée.

Le `try` doit donc s'ouvrir **juste après la création de la ressource**, et pas juste avant `use()` :

```
testUser: async ({ userService, walletService }, use) => {
  const user = await userService.create({ email });
  try {
    await userService.validateKyc(user.id);
    await walletService.deposit(user.id, 10_000);
    await use(await userService.get(user.id));
  } finally {
    await userService.delete(user.id).catch((error) => {
      console.warn(`Cleanup failed for user ${user.id}`, error);
    });
  }
},
```

Formulé autrement : **le runner couvre l'échec du test, votre `try` couvre l'échec de votre propre setup**. Ce sont deux périmètres distincts, et les confondre conduit à du code défensif inutile sur les fixtures simples, tout en laissant les fixtures composées sans protection là où elles en auraient besoin.

Le `.catch` sur la suppression est délibéré, mais il ne doit pas être vide. Si le nettoyage échoue, on ne veut pas que cette erreur secondaire masque la cause réelle de l'échec du test — mais l'avaloir complètement transforme des ressources orphelines en fantômes silencieux, invisibles jusqu'à ce qu'ils saturent une contrainte d'unicité ailleurs. Un nettoyage raté ne doit généralement pas masquer la cause principale du test, mais il doit rester observable : un `console.warn` minimal comme ci-dessus, ou mieux, une attache au rapport de test.

5. Les fixtures natives

Playwright en fournit un jeu prêt à l'emploi. Ce sont celles que vous utilisez dans `{ page }` depuis le début.

Fixture	Type	Scope	À savoir
<code>page</code>	<code>Page</code>	test	Une page isolée pour ce test. La plus utilisée.
<code>context</code>	<code>BrowserContext</code>	test	<code>page</code> en dépend. Pour deux onglets dans un test, créez-les depuis <code>context</code> .
<code>browser</code>	<code>Browser</code>	worker	Partagé entre les tests d'un worker. Ne le modifiez jamais depuis un test.
<code>browserName</code>	<code>string</code>	worker	'chromium', 'firefox' ou 'webkit'. Utile pour les skips conditionnels.
<code>request</code>	<code>APIRequestContext</code>	test	Client HTTP isolé, pour le testing API-first.
<code>playwright</code>	—	worker	Donne accès à <code>playwright.request.newContext()</code> , sans navigateur.

La hiérarchie se lit comme un immeuble : `browser` est le bâtiment, partagé par les locataires d'un même worker. `context` est l'appartement, avec ses propres clés, c'est-à-dire ses cookies et son stockage. `page` est la pièce. Quand un test finit, l'appartement et la pièce sont détruits. L'immeuble reste debout.

6. Les scopes

Par défaut une fixture est test-scoped : une instance neuve par test, isolation garantie. Le scope worker permet de partager une ressource entre tous les tests qu'un même worker exécute.

Scope	Durée de vie	À utiliser pour
Test (défaut)	Un test	Données métier, Page Objects, tout ce qui est modifié
Worker	Tous les tests d'un worker	Ressources coûteuses sans état métier partagé entre tests : jeton admin, connexion base, client API
Auto	Un test ou un worker, sans être demandée	Instrumentation transversale (section 7)

La règle tient en une ligne : **worker pour les ressources coûteuses dont l'état peut être partagé sans créer de dépendance entre les tests, test pour tout le reste.** Le critère n'est pas une immutabilité littérale — un client DB ou un service peut techniquement évoluer

intérieurement (pool de connexions, cache interne) sans que ce soit un problème. Ce qui compte, c'est l'absence d'état métier qui contaminerait les tests suivants.

La contrainte de dépendance

Une fixture test-scoped peut dépendre d'une fixture worker-scoped. L'inverse est interdit : une fixture worker-scoped ne peut pas dépendre d'une fixture test-scoped, puisqu'elle lui survit.

Worker-scoped ne veut pas dire créée au démarrage

C'est le piège le plus contre-intuitif du système. Une fixture worker-scoped reste **lazy** : elle est créée à la première demande, pas au lancement du worker. Si seul le troisième test du worker en a besoin, elle n'existe qu'à partir du troisième test.

En revanche elle est bien détruite à la fin du worker, pas à la fin du test qui l'a demandée.

Worker-scoped = **détruite** à la fin du worker. Mais **créée** à la première demande.

La bonne façon d'écrire une fixture worker

Voici l'implémentation que je recommande pour l'authentification admin. Le point important est qu'elle ne lance aucun navigateur.

```
adminToken: [async ({ playwright }, use) => {
  const api = await playwright.request.newContext({ baseURL: process.env.API_URL });
  try {
    const res = await api.post('/auth/admin-login', {
      data: { email: process.env.ADMIN_EMAIL!, password: process.env.ADMIN_PASSWORD! },
    });
    const { token } = await res.json();
    await use(token);
  } finally {
    await api.dispose();
  }
}, { scope: 'worker' }]],
```

Beaucoup d'implémentations, y compris l'exemple officiel, ouvrent un `browser.newContext()` et une page pour faire cet appel de login. C'est un navigateur complet démarré pour un POST en JSON. Avec quatre workers, ce sont quatre navigateurs inutiles à chaque run.

`playwright.request` est un client HTTP pur, disponible au scope worker, et il fait le même travail pour une fraction du coût.

Second point de conception : la fixture expose **le jeton**, pas un client pré-authentifié. Un `APIRequestContext` fige ses en-têtes à la création, donc exposer un client authentifié oblige à en instancier un deuxième après le login. Exposer une chaîne évite ce doublon, garde la fixture à une seule responsabilité, et laisse chaque service décider de ses propres en-têtes.

Le piège

Mettre une donnée mutable en worker-scoped détruit l'isolation. Deux tests du même worker partagent alors le même enregistrement, et l'ordre d'exécution devient significatif. C'est une des sources de flakiness les plus difficiles à diagnostiquer, parce que le symptôme change avec le nombre de workers, donc entre le poste local et la CI.

Traitez les fixtures worker-scoped comme **en lecture seule** depuis un test.

7. Les fixtures automatiques

Une fixture `auto` s'exécute pour chaque test sans qu'il ait à la demander. C'est le bon outil pour l'instrumentation transversale : capture d'erreurs console, mesure de durée, journalisation réseau.

```
consoleGuard: [async ({ page }, use, testInfo) => {
  const errors: string[] = [];
  page.on('console', msg => { if (msg.type() === 'error') errors.push(msg.text()); });
  page.on('pageerror', err => errors.push(err.message));

  await use();

  if (errors.length > 0) {
    await testInfo.attach('console-errors', {
      body: errors.join('\n'),
      contentType: 'text/plain',
    });
  }
}, { auto: true }],
```

Deux choses à noter. D'abord `use()` est appelé **sans argument**, parce que cette fixture n'expose aucune valeur : elle est typée `void` et n'existe que pour ses effets de bord. L'appel reste obligatoire, c'est lui qui délimite le setup du teardown et qui laisse le test s'exécuter.

Ensuite le choix de conception : la fixture **attache** les erreurs au rapport, elle n'assert pas. Une fixture qui fait échouer un test sur une erreur console transforme chaque warning de librairie tierce en faux positif, et l'équipe finit par la désactiver. En attachant, l'information remonte dans

le rapport HTML sans polluer le résultat. Si l'équipe décide ensuite d'être stricte, l'assertion se met dans un test dédié, pas dans l'infrastructure.

Auto plus worker

Combiner les deux options donne l'équivalent d'un `beforeAll` et `afterAll` globaux :

```
seedDatabase: [async ({}, use) => {
  await seed();
  await use();
  await cleanup();
}, { auto: true, scope: 'worker' }],
```

À retenir : les fixtures auto **worker-scoped** s'exécutent avant `beforeAll`. Les fixtures auto **test-scoped** s'exécutent après.

8. Les fixtures paramétrables

Une fixture déclarée avec `option: true` reçoit une valeur par défaut, redéfinissable depuis la configuration par projet. C'est ce qui permet de faire tourner la même suite sur plusieurs environnements sans variable globale.

```
// fixtures/base.ts
export type TestOptions = {
  apiBaseUrl: string;
  blockedDomains: RegExp[];
};

export const test = base.extend<TestOptions & Fixtures>({
  apiBaseUrl: ['https://api.staging.local', { option: true }],
  blockedDomains: [/google-analytics\.com/, /hotjar\.com/], { option: true }],
});
```

```
// playwright.config.ts
import { defineConfig } from '@playwright/test';
import type { TestOptions } from './fixtures/base';

export default defineConfig<TestOptions>({
  projects: [
    { name: 'staging', use: { apiBaseUrl: 'https://api.staging.local' } },
    { name: 'preprod', use: { apiBaseUrl: 'https://api.preprod.local' } },
  ],
});
```

Le typage est le point clé. `defineConfig<TestOptions>` fait remonter l'autocomplétion et la vérification jusque dans le fichier de configuration. Une faute de frappe sur `apiBaseUrl` devient une erreur de compilation, pas une suite qui tape sur le mauvais environnement pendant trois semaines.

Le piège du tableau

Si la valeur d'une option est un tableau, il faut l'envelopper dans un tableau supplémentaire au moment de la fournir. La documentation le signale explicitement, et l'oubli est silencieux : Playwright interprète le tableau comme la syntaxe tuple `[valeur, options]` et ne récupère que le premier élément.

```
✗ test.use({ persons: [{ name: 'Alice' }, { name: 'Bob' }] });
✓ test.use({ persons: [[{ name: 'Alice' }, { name: 'Bob' }], { scope: 'test' }] });
```

9. Les timeouts de fixture

Le temps d'exécution d'une fixture est **inclus dans le timeout du test**. Si votre test a trente secondes et que la fixture en consomme vingt-cinq à peupler une base, il reste cinq secondes au test lui-même. L'échec qui en résulte pointe le test, alors que la cause est dans le setup.

La solution est un timeout dédié :

```
seedDatabase: [async ({}, use) => {
  await seedLargeDataset();
  await use();
}, { timeout: 60_000 }],
```

La fixture a alors son propre budget, et le timeout du test ne couvre plus que le code du test.

Les fixtures worker-scoped disposent déjà d'un timeout séparé, égal au timeout de test par défaut. Vous pouvez le redéfinir de la même façon.

10. Redéfinir une fixture native

On peut surcharger les fixtures fournies par Playwright, y compris `page` et `context`. C'est le moyen le plus propre d'appliquer un comportement à toute la suite.

```
page: async ({ page, blockedDomains }, use) => {
  for (const domain of blockedDomains) {
    await page.route(domain, route => route.abort());
  }
  await use(page);
},
```

Deux bénéfices. Les tests gagnent plusieurs centaines de millisecondes chacun, ce qui se voit sur une régression complète. Et surtout, la suite ne dépend plus de la disponibilité de services tiers sur lesquels personne n'a la main. Un test qui échoue parce qu'un CDN d'analytics répond en trois secondes mesure autre chose que ce qu'il prétend mesurer.

Noter que la liste de domaines passe par une option (section 8) plutôt que d'être codée en dur. Cette liste vieillit à chaque outil que le marketing ajoute au site, elle n'a pas sa place dans le code des fixtures.

Quand vous surchargez une fixture native, vous devez toujours appeler `use()` avec elle.

Vous enrichissez le comportement, vous ne le remplacez pas. Et la surcharge s'applique à tous les tests qui importent votre `test` custom : assurez-vous que c'est bien l'intention.

11. L'ordre d'exécution

Trois règles suffisent à tout déduire.

1. **Dépendance.** Si A dépend de B, alors B est monté avant A et démonté après A.
2. **Lazy.** Une fixture non-auto n'est créée que si un test la demande.
3. **Scope.** Les test-scoped sont détruites après chaque test, les worker-scoped après le dernier test du worker.

Ce qui donne, en détail :

Setup

1. `browser` et les fixtures worker-scoped natives
2. Fixtures worker-scoped `auto`
3. `beforeAll`
4. Fixtures test-scoped `auto`
5. Fixtures test-scoped demandées par `beforeEach`
6. `beforeEach`
7. Fixtures test-scoped demandées par le test
8. **Le test**

Teardown, en ordre inverse

1. `afterEach`
2. Fixtures test-scoped, dans l'ordre inverse de leur setup
3. `afterAll`, après le dernier test du worker
4. Fixtures worker-scoped
5. `browser`

Une fixture qui n'est jamais demandée n'est jamais exécutée, ni au setup ni au teardown.

12. Les pièges de `beforeAll` et `afterAll`

C'est le point où je vois le plus d'erreurs, y compris chez des équipes expérimentées, parce que le nom du hook suggère quelque chose de faux.

`beforeAll` ne s'exécute pas une fois pour la suite. La formulation exacte est celle de la documentation : il s'exécute une fois par **processus worker**, avant tous les tests de la portée où il est déclaré. Cette portée est le fichier, ou le groupe `describe` s'il y est écrit. Conséquence pratique : un même worker qui enchaîne trois fichiers exécutera trois fois le `beforeAll`, un par fichier. Un setup qui crée une ressource censée être unique tombera sur une contrainte d'unicité dès le deuxième.

Un échec peut le relancer. Playwright met fin au processus worker après certains échecs et en démarre un nouveau pour la suite de la file, retries compris. Ce nouveau worker n'hérite d'aucun état et rejoue son propre `beforeAll`. Le nombre d'exécutions du setup dépend donc du nombre d'échecs, ce qui est exactement la propriété qu'on ne veut pas dans un setup.

Sa donnée est partagée. Tout ce qui est créé dans un `beforeAll` est visible par tous les tests de la portée. Si l'un d'eux la modifie, les suivants sont pollués. Le test qui échoue n'est jamais celui qui a causé le problème, ce qui rend le diagnostic long.

`afterAll` peut ne jamais s'exécuter. Si le processus worker meurt, ou si le job CI est interrompu, le nettoyage saute silencieusement. Contrairement au teardown d'une fixture, il ne bénéficie d'aucune garantie du runner en cas de sortie brutale.

Ce qu'il faut faire à la place

`beforeAll` n'est pas équivalent, on vient de le voir. `globalSetup` existe aussi et s'exécute bien une seule fois — mais il tourne hors du modèle fixtures/reporting de Playwright : pas de traces, pas d'accès aux fixtures normales, pas de ligne dans le rapport HTML. Pour un setup global

intégré au modèle Playwright, observable dans les rapports et pouvant utiliser les fonctionnalités normales du runner, privilégie un projet de setup avec `dependencies` plutôt que `globalSetup`.

```
// playwright.config.ts
projects: [
  { name: 'setup', testMatch: /global\.setup\.ts/ },
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'] },
    dependencies: ['setup'], // n'exécute rien tant que setup n'a pas réussi
  },
],
```

Si le projet `setup` échoue, les projets qui en dépendent ne démarrent pas du tout. C'est préférable à une suite qui part quand même et produit deux cents échecs dont la cause commune est à la première ligne du setup.

Pour tout ce qui relève de la donnée métier, en revanche, une fixture remplace le hook avantageusement.

`beforeEach` reste parfaitement légitime pour une navigation commune à tous les tests d'un fichier. Ce n'est pas un hook à bannir, c'est un hook à ne pas surcharger.

13. Composer et assainir

`mergeTests`

Dès que les fixtures dépassent un fichier, il faut les fusionner explicitement. C'est le cas typique quand une équipe maintient un module de données et un autre d'accessibilité.

```
import { mergeTests } from '@playwright/test';
import { test as dbTest } from './fixtures/database';
import { test as a11yTest } from './fixtures/a11y';

export const test = mergeTests(dbTest, a11yTest);

test('checkout is accessible', async ({ database, a11y, page }) => { /* ... */ });
```

`box` et `title`

Chaque fixture apparaît comme une étape dans le rapport HTML et le Trace Viewer. Sur une suite avec dix fixtures utilitaires, ça noie les étapes métier.

```
consoleGuard: [handler, { auto: true, box: true }], // masquée du rapport
tradingUser: [handler, { title: 'Trader KYC validé' }], // libellé lisible
```

Le réflexe : `box` pour l'infrastructure, `title` pour les fixtures métier dont le nom technique ne parle pas au lecteur du rapport.

14. La fixture de base complète

Voici l'assemblage, sur l'application fil rouge du livre. La structure suit une règle simple : les Page Objects encapsulent l'interface, les services encapsulent l'API, les fixtures de données orchestrent la création et la suppression.

```
// fixtures/base.ts
import { test as base } from '@playwright/test';
import { randomUUID } from 'node:crypto';
import { TradePage } from '../pages/TradePage';
import { WalletPage } from '../pages/WalletPage';
import { UserService } from '../services/UserService';
import { WalletService } from '../services/WalletService';

type TradingUser = { id: number; email: string; token: string; balance: number };

type WorkerFixtures = {
  adminToken: string;
};

type Fixtures = {
  tradePage: TradePage;
  walletPage: WalletPage;
  userService: UserService;
  walletService: WalletService;
  tradingUser: TradingUser;
  consoleGuard: void;
};

export const test = base.extend<Fixtures, WorkerFixtures>({

  // ----- worker -----
  adminToken: [async ({ playwright }, use) => {
    /* voir section 6 */
  }, { scope: 'worker' }],

  // ----- services -----
  userService: async ({ request, adminToken }, use) => {
    await use(new UserService(request, adminToken));
  },
```

```

walletService: async ({ request, adminToken }, use) => {
  await use(new WalletService(request, adminToken));
},

// ----- page objects -----
tradePage: async ({ page }, use) => { await use(new TradePage(page)); },
walletPage: async ({ page }, use) => { await use(new WalletPage(page)); },

// ----- données -----
tradingUser: async ({ userService, walletService }, use) => {
  const user = await userService.create({ email: `trader-${randomUUID()}@test.local` });
  // le try s'ouvre dès que la ressource existe, cf. section 4
  try {
    await userService.validateKyc(user.id);
    await walletService.deposit(user.id, 10_000);
    await use(await userService.get(user.id));
  } finally {
    await userService.delete(user.id).catch((error) => {
      console.warn(`Cleanup failed for user ${user.id}`, error);
    });
  }
},

// ----- instrumentation -----
consoleGuard: [async ({ page }, use, testInfo) => {
  /* voir section 7 */
}, { auto: true, box: true }],
});

export { expect } from '@playwright/test';

```

Un détail qui compte : la génération de l'identifiant

On voit souvent ``user-${Date.now()}@test.com``. Deux problèmes. En parallèle, deux workers qui créent un utilisateur dans la même milliseconde produisent le même email, et la collision se manifeste par un échec aléatoire sur une contrainte d'unicité. Et le domaine `test.com` existe réellement, ce qui expose à envoyer de vrais emails depuis un environnement de recette.

`randomUUID()` et un TLD réservé comme `.local` ou `.test` règlent les deux d'un coup.

15. Ce que ça donne côté test

```

import { test, expect } from '../../fixtures/base';

test('should debit the wallet when a market order is executed @smoke',
  async ({ tradePage, tradingUser, walletService }) => {

```

```

const initialBalance = tradingUser.balance;

await tradePage.goto('AAPL');
const orderId = await tradePage.placeMarketOrder({ symbol: 'AAPL', quantity: 10 });

// La vérité est côté backend, pas à l'écran
const order = await walletService.waitForOrderSettlement(orderId);
expect(order.status).toBe('SETTLED');

const wallet = await walletService.getBalance(tradingUser.id);
expect(wallet.balance).toBeLessThan(initialBalance);
});

```

Le test ne contient plus aucune mécanique. Pas de création d'utilisateur, pas de login, pas de nettoyage, pas de sélecteur. Il lit comme la description métier du scénario, et c'est le critère auquel je juge un framework : **est-ce qu'une personne qui ne connaît pas le code comprend ce que le test vérifie en le lisant une fois.**

Noter aussi que l'assertion finale interroge le backend et non l'affichage. Une interface peut afficher le bon solde alors que la transaction a été enregistrée deux fois.

16. Les cinq anti-patterns

Tout mettre dans `beforeEach`. Setup et cleanup séparés, exécution pour tous les tests même ceux qui n'en ont pas besoin, non réutilisable entre fichiers. Convertissez en fixture.

La fixture fourre-tout. Une fixture qui crée l'utilisateur, le panier, le produit et le paiement. Le test qui voulait juste un utilisateur se tape tout le setup, on ne peut plus tester le panier vide, et un échec est indébogable. Une fixture par concept, elles se composent si besoin.

Des assertions dans les fixtures. Une fixture prépare, un test vérifie. Un `expect` qui échoue dans une fixture produit un message qui ne désigne pas le test, et le test ne montre plus ce qu'il contrôle. Si vous devez vérifier que le setup a fonctionné, laissez l'erreur naturelle remonter ou attachez au rapport.

Muter une fixture `worker-scoped`. Le test suivant voit l'état modifié. Les tests passent seuls et échouent en parallèle. Traitez-les comme en lecture seule.

Ne pas typer. `base.extend({ ... })` sans paramètre de type vous prive de l'autocomplétion et de la vérification. Déclarez toujours `type Fixtures = { ... }` puis `base.extend<Fixtures>({ ... })`.

17. Récapitulatif des pièges

Symptôme	Cause	Correction
Données orphelines malgré un teardown écrit	La fixture échoue avant d'atteindre <code>use()</code>	Ouvrir le <code>try</code> juste après la création de la ressource
Échec aléatoire selon le nombre de workers	Donnée mutable en scope worker	Repasser la fixture en test-scoped
Contrainte d'unicité violée en parallèle	<code>Date.now()</code> comme source d'unicité	<code>randomUUID()</code>
Setup rejoué plusieurs fois	<code>beforeAll</code> pris pour un setup global	Projet de setup avec <code>dependencies</code>
Un test échoue à cause du précédent	Donnée créée en <code>beforeAll</code> et modifiée	Fixture test-scoped
Suite lente sans raison apparente	Setup en <code>beforeEach</code> pour tous les tests	Fixture lazy, créée à la demande
Timeout sur un test dont le code est court	Fixture lente incluse dans le budget du test	Timeout dédié sur la fixture
Option tableau qui ne remonte qu'un élément	Tableau interprété comme syntaxe tuple	Envelopper dans un tableau supplémentaire
Rapport HTML noyé sous les étapes	Fixtures utilitaires visibles	<code>{ box: true }</code>
Échecs liés à des services tiers	Télémetrie non bloquée	Surcharge de la fixture <code>page</code>

18. Version entretien

Les fixtures Playwright sont un système d'injection de dépendances : le test déclare ce dont il a besoin, le framework le résout, l'injecte et le nettoie. Elles sont typées, lazy et composables, donc une fixture n'est instanciée que si un test la réclame, et elle peut dépendre d'une autre.

Le cycle de vie tient dans un seul bloc, le setup avant `use()` et le teardown après, là où un hook l'éclate sur deux endroits. Et le teardown est garanti par le runner même quand le test échoue ou expire, ce qu'un `afterAll` ne garantit pas. La seule zone à couvrir soi-même,

c'est l'échec d'une étape de setup après création de la ressource : là un `try / finally` ouvert dès la création évite les données orphelines.

Le scope worker permet en plus de partager une ressource coûteuse et immuable entre les tests d'un worker, à condition de la traiter en lecture seule. En pratique je garde `beforeEach` pour une navigation commune, et je passe tout le reste en fixtures.

Extrait du chapitre 13.3. Le livre couvre l'architecture complète d'un framework Playwright : arborescence, Page Objects scopés composant, services API, stratégie de données de test, configuration commentée et intégration CI/CD.

Sortie prévue en septembre 2026.

© 2026 Daura Rady. Le texte de cet extrait est diffusé sous licence [CC BY-NC-ND 4.0](https://creativecommons.org/licenses/by-nc-nd/4.0/) : attribution obligatoire, pas d'usage commercial, pas de modification. Les extraits de code sont sous licence MIT.